



绝对是讲的最清楚的-NodeJS模块系统



chenbo

关注

46 人赞同了该文章 >

NodeJS⁺目前有两个系统：一套是CommonJS⁺(简称CJS)，另一套是ECMAScript modules(简称ESM); 本篇内容主要三个话题:

1. CommonJS的内部原理
2. NodeJS平台的ESM模块系统
3. CommonJS与ESM的区别；如何在两套系统进行转换

首先讲讲为什么要有模块系统

为什么要有模块系统

一门好的语言一定要有模块系统，因为它能为我们解决工程中遇到的基本需求

- 把功能进行模块拆分，能够让代码更具有条理，更容易理解，能够让我们单独开发并测试各个子模块的功能
- 能够对功能进行封装，然后再其他模块能够直接引入使用，提高复用性
- 实现封装：只需要对外提供简单的输入输出文档，内部实现能够对外屏蔽，减少理解成本
- 管理依赖关系：好的模块系统能够让开发者根据现有的第三方模块，轻松的构建其他模块。另外模块系统能够让用户简单引入自己想要的模块，并且把依赖链上的模块进行引入

刚开始的时候，JavaScript并没有好的模块系统，页面主要是通过多个script标签引入不同的资源。但是随着系统的逐渐复杂化，传统的script标签模式不能满足业务需求，所以才开始计划定义一套模块系统，有AMD⁺,UMD⁺等等

NodeJS是运行在后台的一门服务端语言，相对于浏览器的html，缺乏script标签来引入文件，完全依赖本地文件系统的js文件。于是NodeJS按照CommonJS规范实现了一套模块系统

2015年ES2015规范⁺发布，到了这个时候，JS才对模块系统有了正式标准，按照这种标准打造的模块系统叫作ESM系统，他让浏览器和服务端在模块的管理方式上更加一致

CommonJS模块

CommonJS规划中有两个基本理念:

- 用户可以通过require函数，引入本地文件系统中的一个模块
- 通过exports和module.exports两个特殊变量，对外发布能力

模块加载器

下面来简单实现一个简单的模块加载器

首先是加载模块内容的函数，我们把这个函数放在私有作用域里边避免污染全局环境，然后eval运行该函数

```
function loadModule(filename, module, require) {  
  const wrappedSrc = `  
    (function (module, exports, require) {  
      ${fs.readFileSync(filename, 'utf-8')}`
```



}

在代码中我们通过同步方法`readFileSync`来读取了模块内容。一般来说，在调用文件系统API时，不应该使用同步版本，但是此处确实是使用了这个方式，Commonjs通过同步操作，来保证多个模块能够安装正常的依赖顺序得到引入

现在在实现`require`函数

```
function require(moduleName) {
  const id = require.resolve(moduleName);
  if (require.cache[id]) {
    return require.cache[id].exports
  }

  // 模块的元数据

  const module = {
    exports: {},
    id,
  }

  require.cache[id] = module;

  loadModule(id, module, require);

  // 返回导出的变量
  return module.exports
}

require.cache = {};
require.resolve = (moduleName) => {
  // 根据ModuleName解析完整的模块ID
}
```

上面实现了一个简单的`require`函数，这个自制的模块系统有几个不走需要解释

1. 输入模块的ModuleName以后，首先要解析出模块的完整路径(如何解析后面会讲到)，然后把这个结果保存在id的变量之中
2. 如果该模块已经被加载过了，会立刻返回缓存中的结果
3. 如果该模板没有被加载过，那么就配置一套环境。具体来说，先创建一个`module`变量，让他包含一个exports的属性。这个对象的内容，将由模块在导出API时所使用的的那些代码来填充
4. 将module对象缓存起来
5. 执行`loadModule`函数，传入刚建立的module对象，通过函数将另外一个模块的内容进行挂载
5. 返回另外模块的导出内容

模块解析算法

在前面提到解析模块的完整路径，我们通过传入模块名，模块解析函数能够返回模块的对应的完整路径，接下来通过路径来加载对应模块的代码，并用这个路径来标识模块的身份。`resolve`函数所用的解析函数主要是处理以下三种情况

- **要加载的是不是文件模块？** 如果moduleName以/开头，那就视为一条绝对路径，加载时只需要安装该路径原样返回即可。如果moduleName以`.`/`开头，那么就当成一条相对路径，这样相对路径是从请求载入该模块的这个目录算起的

- ****要加载的是不是包模块**** 如果没有找到`moduleName`匹配的核心模块，那就从发出加载请求的这个模块开始，逐层向上搜寻名为`node\_modules`的陌路，看看里边有没有能够与`moduleName`匹配的模块，如果有就载入该模块。如果还没有，就沿着目录继续线上走，并在相应的`node\_modules`目录中搜寻，一直到文件系统的根目录

通过这种方式就能实现两个模块依赖不同版本的包，但是仍然能够正常加载

例如以下目录结构:

```
myApp
- index.js
- node_modules
  - depA
    - index.js
  - depB
    - index.js
    - node_modules
      - depA
  - depC
    - index.js
    - node_modules
      - depA
```

在上述例子中虽然`myApp`、`depB`、`depC`都依赖了`depA`但是加载进来的确实不同的模块。比如:

- 在`/myApp/index.js`中，加载的来源是`/myApp/node\_modules/depA`

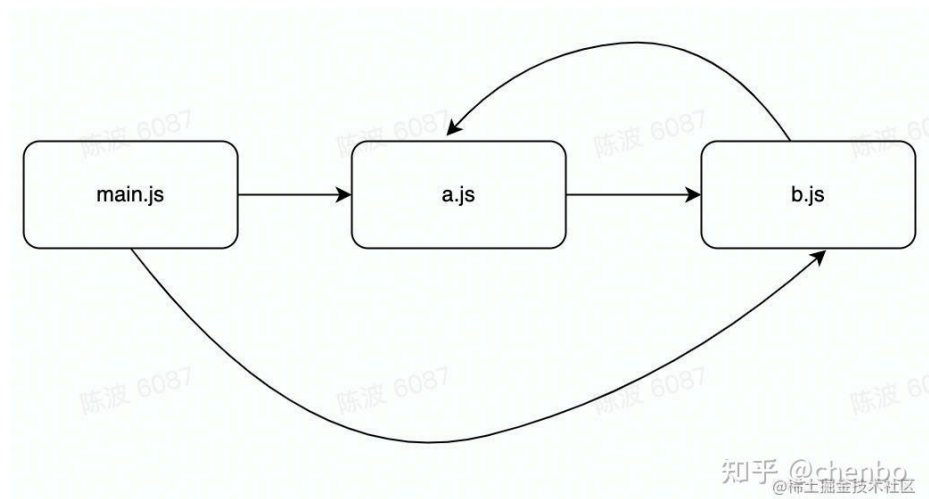
- 在`/myApp/node\_modules/depB/index.js`，加载的是`/myApp/node\_modules/depB/node\_modules/depA`

- 在`/myApp/node\_modules/depC/index.js`，加载的是`/myApp/node\_modules/depC/node\_modules/depA`

NodeJs之所以能够把依赖关系管理好，就因为它背后有模块解析算法这样一个核心的部分，能够管理上千个包，而不会发生冲突或出现版本不兼容的问题

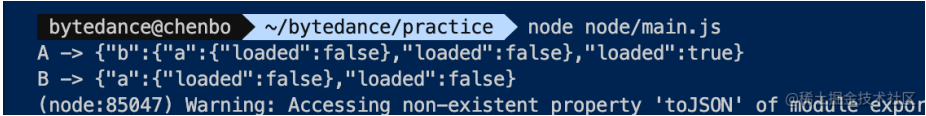
循环依赖

很多人觉得循环依赖是理论上的设计问题，但是这种问题很可能出现在实际项目中，所以应该知道CommonJS如何处理这种情况的。是看之前实现的require函数就能够意识到其中的风险。下面通过一个例子来讲解



```
// a.js
exports.loaded = false;
const b = require('./b');
module.exports = {
  b,
  loaded: true
}
// b.js
exports.loaded = false;
const a = require('./a')
module.exports = {
  a,
  loaded: false
}
// main.js
const a = require('./a');
const b = require('./b');
console.log('A ->', JSON.stringify(a))
console.log('B ->', JSON.stringify(b))
```

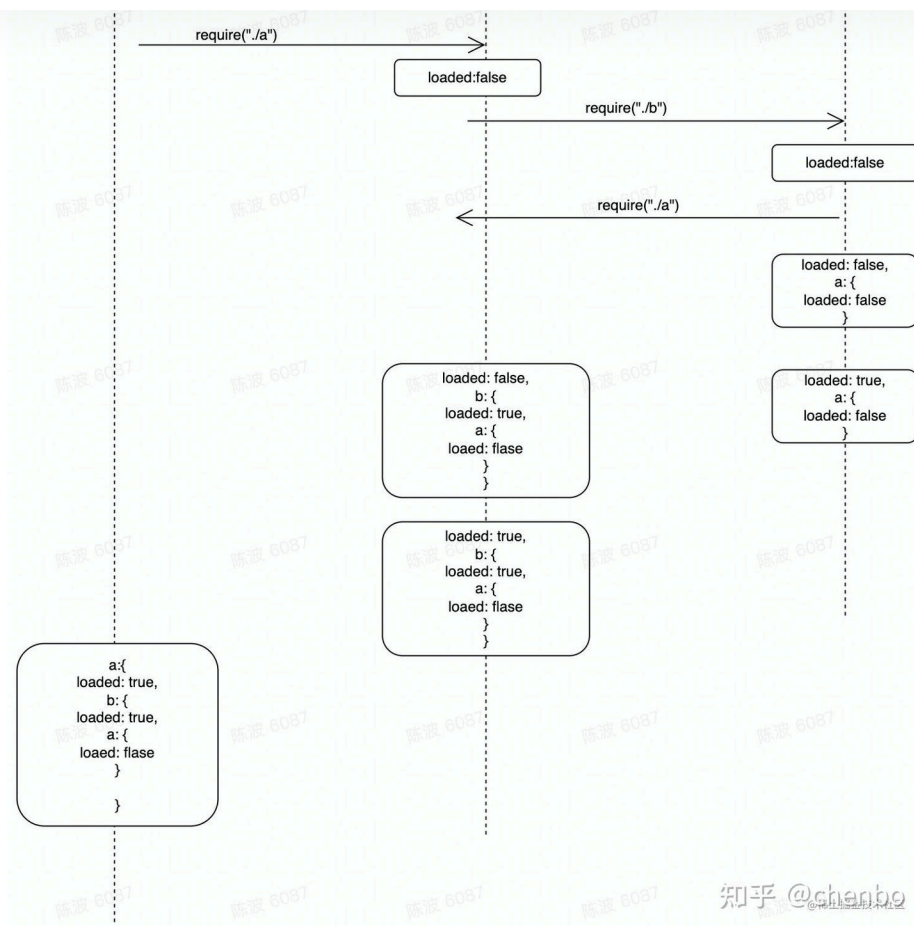
运行`main.js`会得到以下结果



```
bytedance@chenbo ~/bytedance/practice node node/main.js
A -> {"b":{"a":{"loaded":false,"loaded":false},"loaded":true}}
B -> {"a":{"loaded":false},"loaded":false}
(node:85047) Warning: Accessing non-existent property 'toJSON' of module export
```

从结果可以看到，CommonJS在循环依赖所引发的风险。b模块导入a模块的时候，内容并不是完整的，具体来说他只是反应了b.js模块请求`a.js`模块时，该模块所处的状态，而无法反应`a.js`模块最终加载完毕的一个状态

下面用一个示例图来表示这个过程



下面是具体的流程解释

1. 整个流程从main.js开始，这个模块一开始开始导入a.js模块
2. a.js首先要做的，是导出一个名为loaded的值，并把该值设为false
3. a.js模块要求导入b.js模块
4. 与a.js类似，b.js首先也是导出loaded为false的变量
5. b.js继续执行，需要导入a.js
6. 由于系统已经开始处理a.js模块了，所以b.js会把a.js已经导出的内容，立即复制到本模块中
7. b.js会把自己导出的loaded值改为false
8. 由于b已经执行完成，控制权会回到a.js，他会把b.js模块的状态拷贝一份
9. a.js继续执行，修改导出值loaded为true
10. 最后就执行main.js

上面可以看到由于是同步执行，导致b.js导入的a.js模块并不是完整的，无法反应b.js的最终应有的状态。

在上面例子中可以看到，循环依赖所产生的的结果，这对大型项目来说，更加严重。

使用方法就比较简单了，篇幅有限就不在这篇文章中进行讲解了

语句必须要写在最顶层。另外受引用的模块只能使用常量字符串，不能依赖需要运行期动态求值的表达式。

比如我们不能通过下面方式来引入ES模块

```
if (condition) {
  import module1 from 'module1'
} else {
  import module2 from 'module2'
}
```

而CommonJS能够根据条件导入不同的模块

```
let module = null
if (condition) {
  module = require("module1")
} else {
  module = require("module2")
}
```

看起来相对CommonJS更严格了一些，但是正是因为这种静态引入机制，我们能够对依赖关系进行静态分析，去除不会执行的逻辑，这个就叫`tree-shaking`

模块加载过程

要想理解ESM系统的运作原理，以及它处理循环依赖的关系，我们需要明白系统是如何解析并执行Javascript代码

载入模块的各个阶段

解释器的目标是构建一张图来描述所要载入的模块之间的依赖关系，这种图也叫做依赖图。

解释器正是通过这种依赖图，来判断模块的依赖关系，并决定自己应该按照什么顺序去执行代码。例如我们需要执行某个js文件，那么解释器会从入口开始，寻找所有的import语句，如果在寻找过程中又遇到了import语句，那就会以深度优先的方式递归，直到所有的代码都解析完毕。

这个过程可细分为三个过程：

1. 剖析：找到所有的引入语句，并递归从相关文件中加载每个模块的内容
2. 实例化：针对某个导出的实体，在内存中保留一个带名称的引入，但暂且不给他赋值。此时还要根据import和export关键字建立依赖关系，此时不执行js代码
3. 执行：到了这个阶段，NodeJS开始执行代码，这能够让实际导出的实体，能够获得实际的取值

在CommonJS中，是边解析依赖，一边执行文件。所以当看到require的时候，就代表前面的代码已经执行完成。因为require操作不一定要在文件开头，而是可以出现在任务地方

但是ESM系统不同，这三个阶段是分开的，它必须先把依赖图完整的构造出来，然后才开始执行代码

循环依赖

在之前提到的CommonJS循环依赖的例子，使用ESM的方式进行改造

```
// a.js
import * as bModule from './b.js';
```

```
loaded = true;
// b.js
import * as aModule from './b.js';
export let loaded = false;
export const a = aModule;
loaded = true;
// main.js
import * as a from './a.js';
import * as b from './b.js';
console.log("A =>", a)
console.log("B =>", b)
```

需要注意的是这里不能用`JSON.stringify`方法，因为这里使用了循环依赖

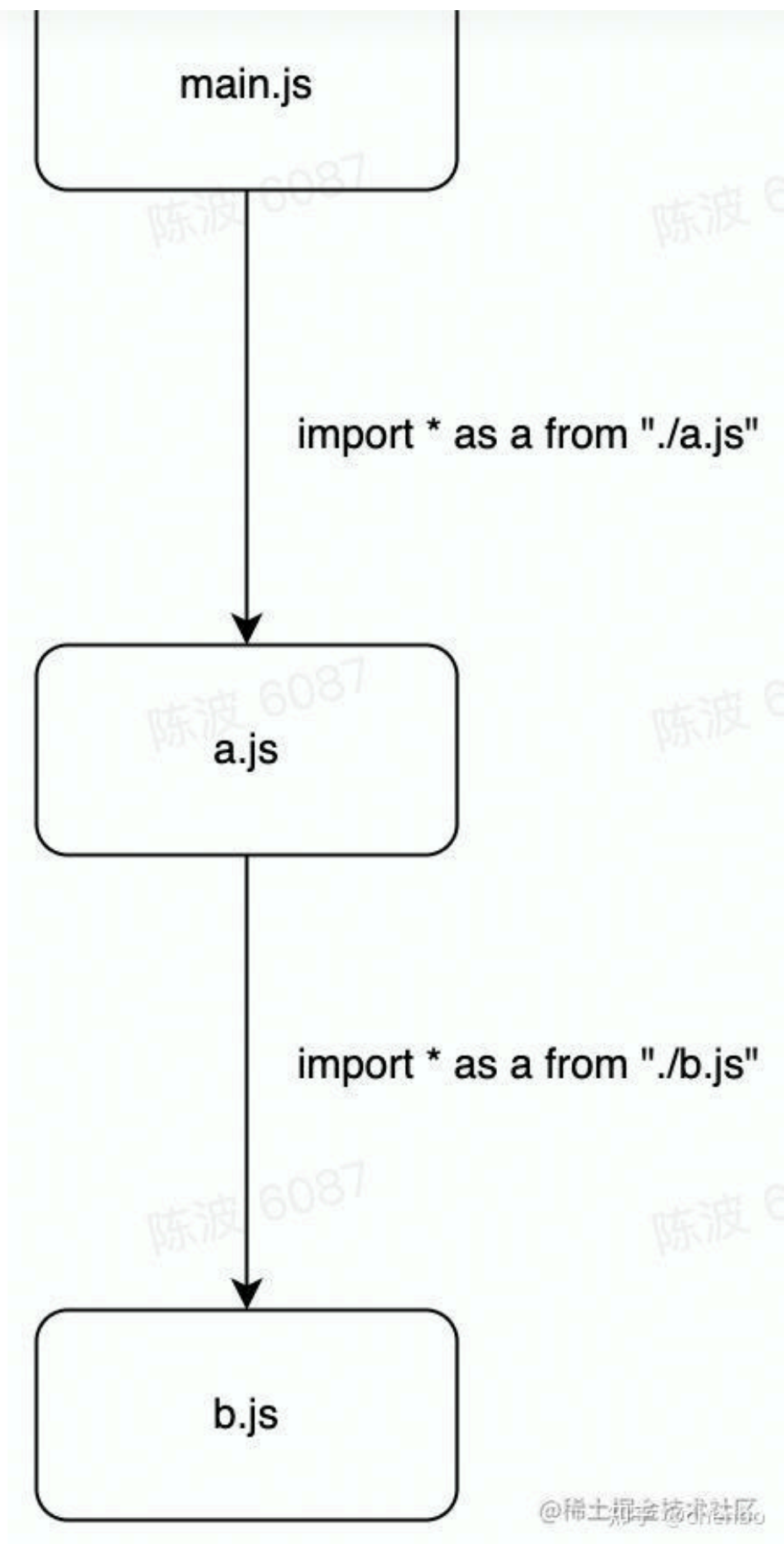
```
A => [Module: null prototype] {
  b: <ref *1> [Module: null prototype] {
    a: [Circular *1],
    loaded: true
  },
  loaded: true
}
B => <ref *1> [Module: null prototype] {
  a: [Circular *1],
  loaded: true
}
```

知乎 @chenbo
@稀土掘金技术社区

在上面执行结果中可以看到a.js和b.js都能够完整的观察到对方，不同与CommonJS，有模块拿到的状态是不完整的状态。

剖析

下面来解析一下其中的过程：



@稀土掘金技术社区

已上图为例：

1. 从main.js开始剖析，首先发现了一条import语句，然后进入a.js
2. 从a.js开始执行，发现了另外一条import语句，执行b.js

4. b.js继续往下执行，发现没有别的import语句。回到a.js之后，也发现没有其他的import语句，然后直接回到main.js入口文件。继续往下执行，发现要求引入b.js，但是这个模块之前被访问过了，因此这条路径不会执行

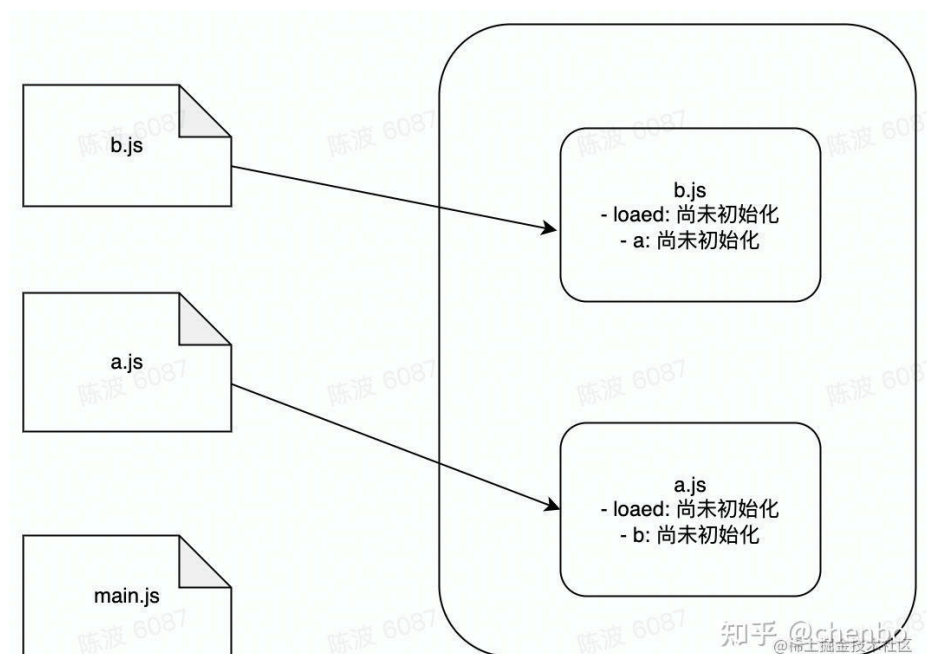
经过深度优先的方式，模块依赖关系图已经形成一个树状图，然后解释器在通过这个依赖图执行代码

在这个阶段，解释器要从入口点开始，开始分析各模块之间的依赖关系。这个阶段解释器只关心系统的import语句，并把这些语句想要引入的模块给加载进来，并以深度优先的方式探索依赖图。按照这种方法遍历依赖关系，得到一种树状的结构

实例化

在这一阶段，解释器会从树状结构的底部开始，逐渐向顶部走。没走到一个模块，它就会寻找该模块所要导出的所有属性，并在内存中构建一张隐射表，以存放此模块所要导出的属性名称与该属性即将拥有的取值

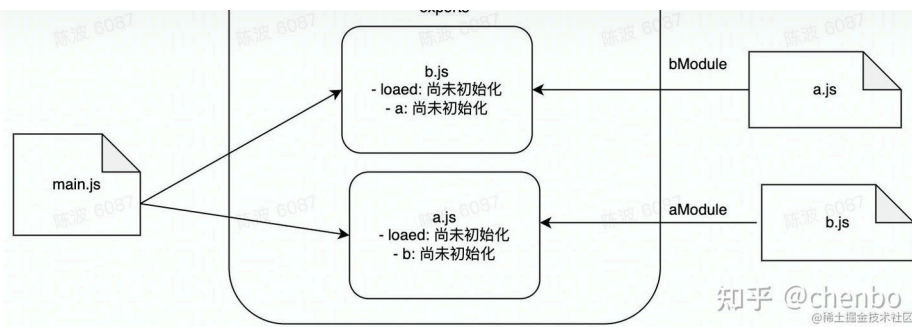
如下图所示：



从上图可以看到，模块是按照什么顺序来实例化的

1. 解释器首先从b.js模块开始，它发现这个模块要导出loaded和a
2. 然后解释器又分析a.js模块，他发现这个模块要导出loaded和b
3. 最后分析main.js模块，他发现这个模块不导出任何功能
4. 实例化阶段所构造的这套exports隐射图，只记录导出的名称与该名称即将拥有的值之间关系，至于这个值本身，既不在本阶段初始化。

走完上述流程后，解析器还需要在执行一遍，这次他会把各模块所导出的名称与引入这些的那些模块关联起来，如下图所示：

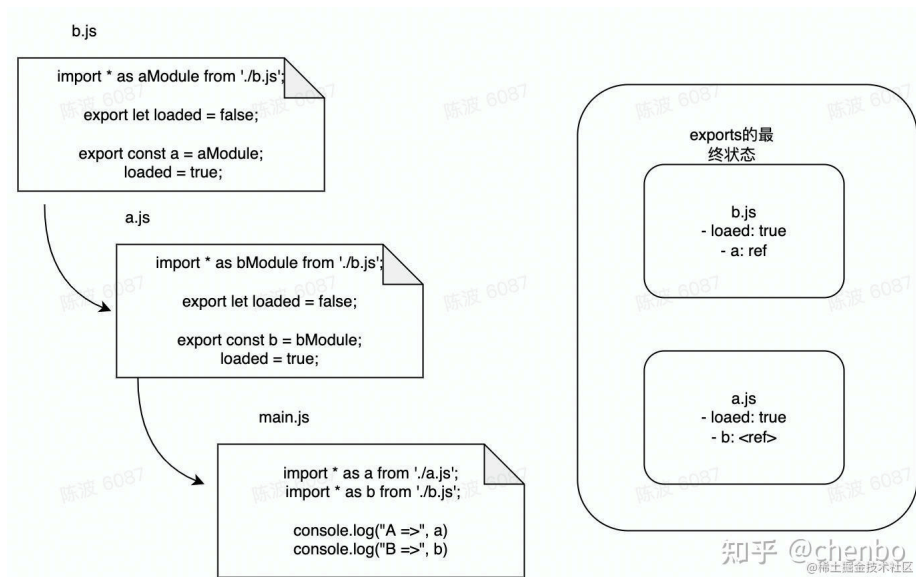


这次的步骤为:

1. 模块b.js要与模块a.js所导出的内容相连接, 这条链接叫作aModule
2. 模块a.js要与模块b.js所导出的内容相连接, 这条链接叫作bModule
3. 最后模块main.js要与模块b.js所导出的内容相连接
4. 在这个阶段, 所有的值并没有初始化, 我们只是建立相应的链接, 能够让这些链接指向相应的值, 至于值本身, 需要等到下一阶段才能确定

执行

这个阶段, 系统终于要执行每份文件里边的代码。他按照后序的深度优先顺序, 由下而上的访问最初那张依赖图, 并逐个执行访问到的文件。在本例中, main.js会放在最后执行。这种执行结果保证了, 程序在运行主逻辑的时候, 各模块所导出的那些值, 全部得到了初始化



以上图具体步骤为:

1. 从b.js开始执行。首先要执行的这行代码, 会把该模块所导出的loaded初始化为false
2. 接下来往下执行, 会把aModule复制给a, 这个时候a拿到的是一个引用值, 这个值就是a.js模块
3. 然后设置loaded的值为true。这个时候b模块所有的值都全部确定了下来
4. 现在执行a.js。首先初始化导出值loaded为false
5. 接下来将该模块导出的b属性值得到初始值, 这个值是bModule的引用

走完这些步骤后，系统就可以正式执行main.js文件，这个时候，各模块所导出的属性全都已经求值完毕，由于系统是通过引用而不是复制来引入模块，所以就算模块之间有循环依赖关系，每个模块还是能够完整看到对方的最终状态

CommonJS与ESM的区别与交互使用

这里讲CommonJS和ESM之间几个重要的区别，以及如何在必要的时候搭配使用这两种模块

ESM不支持CommonJS提供的某些引用

CommonJS提供一些关键引用，不受ESM支持，这包括`require`、`exports`、`module.exports`、`\_\_filename`、`\_\_dirname`。如果在ES模块中使用这些，会到程序发生引用错误的问题。

在ESM系统中，我们可以通过import.meta这个特殊对象来获取一个引用，这个引用指的是当前文件的URL。具体来说，就是通过import.meta.url这种写法，来获取当前模块的文件路径，这个路径类似于`file:///path/to/current\_module.js`。我们可以根据这条路径，构造出`\_\_filename`和`\_\_dirname`所表示的那两条绝对路径：

```
import { fileURLToPath } from 'url';
import { dirname } from 'path';
const __dirname = fileURLToPath(import.meta.url);
const __dirname = dirname(__filename);
```

CommonJS的require函数，也可以通过用下面这种方法，在ESM模块里边进行实现：

```
import { createRequire } from 'module';
const require = createRequire(import.meta.url)
```

现在，就可以在ES模块系统的环境下，用这个`require()`函数来加载`Commonjs`模块

在其中一个模块系统中使用另外一个模块

在上面提到，在ESM模块中使用`module.createRequire`函数来加载`commonJS`模块。除了这个方法，其实还可以通过import语言引入CommonJS模块。不过这种方式只会导出默认导出的内容：

```
import pkg from 'commonJS-module'
import { method1 } from 'commonJS-module' // 会报错
```

但是反过来没办法，我们没办法在`commonJS`中引入`ESM`模块

此外ESM不支持把json文件当成模块进行引入，这在commonjs却可以轻松实现

下面这种import语句，就会报错

```
import json from 'data.json'
```

如果需要引入json文件，还需要借助`createRequire`函数：

```
import { createRequire } from 'module';
const require = createRequire(import.meta.url);
const data = require("./data.json");
console.log(data)
```

本文主要讲解了NodeJS中两种模块系统是如何工作的，通过了解这些原因能够帮忙我们编写避免一些难以排查的问题的bug

编辑于 2022-01-16 23:51

内容所属专栏



前端好好学
用于发布日常博客内容

订阅专栏

前端开发

赞同 46

3 条评论

分享

喜欢

收藏

申请转载



理性发言，友善互动

3 条评论

默认 最新



朱佳伟

这个像webpack 里面的模块

2023-08-04

回复 喜欢



清欢bx

CommonJS 执行js时是包裹了一个(function (module, exports, require) {\$(fs.readFileSync(filename, 'utf-8'))})(module, module.exports, require) 那ESM 是怎么加载和执行js的

2022-11-18

回复 喜欢



扬帆起航

真不错

2022-10-14

回复 喜欢

推荐阅读



几个常见的 NodeJS 误区

A-yon

手把手，教你用nodejs搭建后台最小系统（大量图文）系列一

手把手，教你用nodejs搭建后台最小系统（大量图文）系列一前言：很久以前就想写这么一篇文章了，因为之前都是用其他语言写后台，但是环境配置不好弄，而且占用运行内存也大，所以趁着这段时...

Msgyvcici



Node.js超详细零基础教程(1)——处理GET、POST请求

兔子先生

发表于互联网技术



如何优雅地让多个版本的nodejs共存？

星游会

发表